Videos publicitarios, alojados internamente y renderizados "dentro del juego" es la mejor decisión para un MVP.

Al usar la etiqueta nativa <video> de HTML5 en lugar de un reproductor externo como YouTube, tienes control absoluto sobre los controles de reproducción y puedes bloquear cualquier intento de saltar el anuncio.

Para mantener la inmersión, llamaremos a esta sección el "Paddock de Patrocinadores" o "El Autocine".

1. Diseño de UI/UX (El Autocine)

Entorno Visual: Un fondo oscuro (ej. un garaje con luces de neón desenfocadas). En el centro, una "pantalla gigante" o cartelera digital.

Interacción: El usuario no ve la barra de progreso típica de YouTube. Solo hay un botón de "Ver Tráiler por 100 CR". Al presionarlo, el video ocupa el centro de la pantalla, oscureciendo aún más el fondo.

Bloqueo de UI: Mientras el video se reproduce, no hay botón de pausa, ni de adelantar, ni de cerrar. Si el usuario quiere salir, pierde el progreso.

Frontend: Bloqueo del Reproductor en React

El mayor desafío en el frontend es evitar que el usuario manipule el video usando la consola del navegador (por ejemplo, inyectando document.querySelector('video').playbackRate = 10 para que el video de 30 segundos termine en 3).

Componente SponsorArena.jsx :

```
import { useRef, useState, useEffect } from 'react';

import { useGarageStore } from '../store/useGarageStore';


export default function SponsorArena({ videoSrc, durationSeconds, trailerId }) {

  const videoRef = useRef(null);

  const [isPlaying, setIsPlaying] = useState(false);

  const [sessionId, setSessionId] = useState(null);
```

```
const addCredits = useGarageStore((state) => state.addCreditsFromTrailer);

// Regla Anti-Cheat 1: Pausar si el usuario cambia de pestaña
useEffect(() => {
  const handleVisibilityChange = () => {
    if (document.hidden && videoRef.current) {
      videoRef.current.pause();
    } else if (!document.hidden && isPlaying && videoRef.current) {
      videoRef.current.play();
    }
  };
  document.addEventListener("visibilitychange", handleVisibilityChange);
  return () => document.removeEventListener("visibilitychange", handleVisibilityChange);
}, [isPlaying]);

const startTrailer = async () => {
  // 1. Pedir permiso al servidor ANTES de reproducir
  const response = await fetch('/api/sponsors/start-session', {
    method: 'POST',
    body: JSON.stringify({ trailer_id: trailerId })
  });
  const data = await response.json();

  setSessionId(data.session_id); // Guardamos el ticket del servidor
  setIsPlaying(true);
  videoRef.current.play();
```

```
};

const handleVideoEnd = async () => {
  setIsPlaying(false);

  // 2. Reclamar recompensa enviando el ticket
  const response = await fetch('/api/sponsors/claim', {
    method: 'POST',
    body: JSON.stringify({ session_id: sessionId })
  });

  if (response.ok) {
    addCredits(100); // Actualización optimista en Zustand
    alert("¡Tráiler completado! +100 CR");
  } else {
    alert("Error en la validación. Tráiler no válido.");
  }
};

return (
  <div className="relative flex justify-center items-center h-screen bg-black/90">
    {/* Reproductor bloqueado (sin el atributo 'controls') */}
    <video
      ref={videoRef}
      src={videoSrc}
      onEnded={handleVideoEnd}
```

```
        className={`w-full max-w-3xl rounded-lg shadow-[0_0_50px_rgba(255,255,255,0.1)]
${isPlaying ? 'opacity-100' : 'opacity-50'}`}

        disablePictureInPicture

        controlsList="nodownload nofullscreen noremoteplayback"

      />



      {!isPlaying && (

        <button

          onClick={startTrailer}

          className="absolute z-10 px-8 py-4 bg-yellow-400 text-black font-bold uppercase
tracking-widest rounded shadow-lg hover:bg-yellow-300 transition"

        >

          Ver Patrocinador (100 CR)

        </button>

      )}

    </div>

  );

}
```

////////////——————————-///////////////——————————/////////////////

Backend: La Barrera de Tiempo Real (Python/SQLite)

Si un usuario malintencionado descubre la URL /api/sponsors/claim, podría usar Postman para enviar peticiones falsas y llenarse de créditos. El servidor debe ser la única fuente de verdad respecto al paso del tiempo.

Flujo en FastAPI (economy.py)

1. POST /start-session: El usuario dice "voy a empezar a ver".

El servidor anota en SQLite: SessionID: xyz, User: Martin, Trailer_Duration: 30s, Start_Time: 13:50:00.

Retorna el SessionID al cliente.

2. POST /claim: El usuario dice "ya terminé".

El servidor busca la sesión xyz.

Compara la hora actual (13:50:31) con el Start_Time (13:50:00).

Diferencia = 31 segundos.

¿31s es mayor o igual a los 30s que dura el tráiler? SÍ.

Resultado: Suma los créditos a la billetera de usuario, marca la sesión como COMPLETED (para que no pueda reutilizar el ticket) y cierra la transacción.

¿Qué pasa si el usuario acelera el video en el navegador a x10 de velocidad?

El video terminará en 3 segundos. El cliente enviará el /claim.

El servidor restará la hora actual (13:50:03) menos el inicio (13:50:00).

Diferencia = 3 segundos.

¿3s es mayor a 30s? NO.

Resultado: El servidor devuelve Error 403 (Fraude), la sesión se anula y no se entregan créditos.

Con este diseño bidireccional, tienes una interfaz inmersiva, limpia de controles molestos de YouTube, y una lógica de backend completamente blindada para presentar ante cualquier jurado técnico.